



The Lazy Developer

20

MODULE

Using AI Agents

Learn how to leverage autonomous AI agents to streamline workflows and enhance productivity.

Table of Contents

- 01. Role of AI Agents (18 min)
- 02. Different platforms with different approaches (18 min)

Role of AI Agents

Let's talk about something that's fundamentally changing how software gets built with AI agents. If you've been using ChatGPT or Claude to help you write code, you've already dipped your toes in. But agents? They're the deep end of the pool, and honestly, it's a lot more fun over here. Since approx Decemeber of 2025, Claude Code and GPT Codex took a turn with their own implementations of agentic coding using tools, building, checking, testing, and delivering. All of this while you can fire off the next agent to work on another task, or check another agent's work. Wild.

What Is an AI Agent vs. a Chatbot?

Here's the simplest way to think about it:

A chatbot is like a waiter at a restaurant. You tell them what you want, they write it down, they bring it to you. If the kitchen is out of salmon, they come back and tell you. You pick something else. Every interaction is a back-and-forth where you are making all the decisions.

An AI agent is like a personal chef who also does the grocery shopping. You say, "I want something Italian for dinner tonight — maybe pasta, but I'm open to ideas." The chef checks what's in your fridge, drives to the store, picks up fresh ingredients, comes back, cooks the meal, plates it beautifully, and serves it. If the store is out of basil, they grab parsley and adjust the recipe. They handle the whole process without coming back to ask you about every little decision.

That's the fundamental difference. A chatbot responds to individual prompts. An agent takes a goal and works toward it autonomously, making decisions along the way.

Key insight: You don't need to understand every step the agent takes. You need to understand what you want built and how to evaluate the result.

The Agentic Loop

Every AI agent follows a pattern that looks something like this:

Perceive !' Plan !' Act !' Observe !' Repeat

Let's break that down with a real example. Say you ask an AI agent to "add a dark mode toggle to my React app":

1. Perceive — The agent reads your project files. It finds your existing components, your CSS setup (Tailwind? Plain CSS? Styled-components?), your theme configuration, and your layout structure.
2. Plan — Based on what it found, the agent decides: "I need to create a ThemeProvider context, add a toggle button component, update the Tailwind config for dark mode, and modify the root layout."

3. **Act** — It starts writing code. It creates new files, modifies existing ones, adds the necessary imports, and wires everything together.
4. **Observe** — The agent checks its work. Did the code compile? Are there any errors? Does the toggle actually switch themes? Some agents can even run your app and verify visually.
5. **Repeat** — If something broke (maybe it forgot to update an import), it goes back to step 1 and fixes it.

This loop is what makes agents so powerful. They don't just give you a code snippet and wish you luck — they iterate until the job is done.

Types of AI Agents in Software Development

Not all agents are created equal. Here are the main types you'll encounter:

Coding Agents

These are the workhorses. They write code, refactor existing code, add features, and fix bugs. Examples include Claude Code's agent mode and Cursor's Composer.

Testing Agents

These agents can write test cases, run your test suite, identify failing tests, and even fix the code that's causing failures. They're like having a QA engineer who never sleeps.

Deployment Agents

These handle the process of getting your code from your computer to a live server. They can configure hosting platforms, set up CI/CD pipelines, and manage environment variables.

Research Agents

These agents can browse documentation, read API references, search for solutions to errors, and synthesize information. They're your tireless research assistant.

Real Examples You Can Use Today

This isn't science fiction. Here are real AI agent tools available right now:

- **Claude Code (Agent Mode)** — Anthropic's terminal-based agent. It can read your entire codebase, create and edit files, run terminal commands, execute tests, and iterate on errors. It works from your command line and has full access to your file system.
- **Cursor's Composer** — Built into the Cursor IDE, Composer can make changes across multiple files simultaneously. It sees your entire project context and can create, edit, and delete files.
- **GitHub Copilot Workspace** — GitHub's agent that can take an issue description and generate a full implementation plan, write the code, and create a pull request.
- **GPT Codex** — OpenAI's cloud-based coding agent. You give it a task, it spins up a cloud environment, writes and tests code, and delivers results.

- Devin — Built by Cognition, Devin is marketed as a fully autonomous software engineer. It can plan, code, debug, and deploy entire features independently.

Each of these tools has different strengths, and we'll compare them in detail in the next section.

What Makes Agents Powerful

Three things separate agents from basic AI chat:

1. Tool Use

Agents can do things, not just say things. They can:

- Read and write files on your computer
- Run terminal commands (like `npm install` or `git commit`)
- Make API calls to external services
- Search the web for documentation
- Execute code and observe the output

2. Memory

Agents maintain context throughout a session. They remember what files they've already read, what changes they've made, and what errors they've encountered. This means they can build on previous steps rather than starting from scratch with each interaction.

3. Multi-Step Reasoning

Agents can break a complex task into smaller steps and execute them in sequence. "Build a user authentication system" becomes a series of planned actions: create the database table, build the API endpoints, create the login form, add session management, wire it all together.

Chat-Based AI vs. Agentic AI

Here's a side-by-side comparison:

Feature	Chat-Based AI	Agentic AI
Input	Single prompt or question	High-level goal or task description
Output	Text response (code snippet, explanation)	Completed work (files created/modified, commands run)
Autonomy	None, it waits for each prompt	High, it makes decisions independently
Error Handling	Shows you the error, you fix it	Detects errors, attempts to fix them automatically
Context	Limited to conversation window	Can read your entire project
Tool Access	None (copy/paste only)	File system, terminal, APIs, browser
Best Use Case	Quick questions, learning concepts	Building features, refactoring, multi-file changes
Speed	Fast for simple answers	Slower but handles complex tasks end-to-end

"Vibe Coding" - A New Way to Build

There's a term that's gained traction in the developer community: vibe coding. The idea is simple, instead of writing every line of code yourself, you describe what you want at a high level and let the AI agent handle the implementation. The term was created by Andrej Karparthy from OpenAI.

Your job shifts from writing code to steering direction:

- "Add a contact form to the homepage with email validation"
- "Make the navigation responsive for mobile"
- "Set up Stripe payment integration with a checkout page"
- "Refactor this component to use TypeScript instead of JavaScript"

You're the architect. The agent is the construction crew. You decide what gets built and what it should look like. The agent figures out the implementation details.

This is incredibly powerful for non-coders because it means you can build real applications without memorizing syntax or understanding every framework API. You focus on what you want, not how to code it.

When to Use Agents vs. Manual Control

Agents are amazing, but they're not always the right tool. Here's a quick guide:

Use an agent when:

- You need changes across multiple files
- You're building a new feature from scratch
- You need to refactor or restructure existing code
- You want to set up a project with boilerplate code
- You're debugging an error you don't understand

Use manual control (chat mode) when:

- You need to understand why something works, not just make it work
- You're making a tiny, specific change to one file
- You want to learn a new concept (agents do the work for you, which isn't always great for learning)
- The change is sensitive (security, payments, data deletion) and you want full oversight
- You want to carefully review each step before it happens

The sweet spot? Start with agents for the heavy lifting, then switch to manual mode when you need precision or understanding. Much of the time, AI will figure out what it needs based on your discussion so in essence you don't have to choose anything.

AI Prompt Examples

Try these prompts in your AI coding if relevant to see what will happen, do you get full agent mode or a simple reply?

```
Look at my project structure and suggest what improvements
could be made to the code organization.
```

```
Add a responsive navigation bar to my app with a hamburger
menu on mobile. Use my existing Tailwind setup.
```

```
I'm getting this error when I run my app: [paste error].
Diagnose the problem and fix it.
```

```
Create a new contact page with a form that collects name,
email, and message. Add form validation and style it to
match the rest of my app.
```

```
Review my codebase and identify any security issues,
performance problems, or best practice violations.
```

Key Takeaways

- AI agents are autonomous — they take a goal and work toward it, making decisions along the way, unlike chatbots that wait for each instruction.
- The agentic loop (perceive, plan, act, observe, repeat) is how agents break down and tackle complex tasks.
- Tool use is the superpower — agents can read files, run commands, make API calls, and interact with your development environment directly.
- Vibe coding lets you focus on what to build while agents handle how to build it which a game-changer for non-coders.
- Agents aren't always the answer — use chat mode for learning, small changes, and sensitive operations where you want full control.
- Real agent tools exist today — Claude Code, Cursor Composer, GitHub Copilot Workspace, GPT Codex, and Devin are all available and getting better fast.

Different Platforms with Different Approaches

So now you know what AI agents are and why they matter. The next question is obvious: which one should you actually use?

The honest answer is that it depends on what you're building, how much control you want, and what your budget looks like. The landscape of AI coding agents has exploded, and each platform has carved out its own niche. Let's walk through the major players, compare them head-to-head, and figure out which one fits your workflow.

The Major AI Agent Platforms

Cursor (Composer Agent Mode)

What it is: Cursor is a code editor (it's actually a fork of VS Code) with AI deeply integrated into every part of the experience. Its standout feature is Composer — an agent mode that can make changes across your entire project.

How it works: You open your project in Cursor like you would in any code editor. When you activate Composer, the agent can see all your files, understand your project structure, and make coordinated changes across multiple files simultaneously. It's like having a pair programmer sitting next to you who can see your entire screen.

Best for: People who want a traditional coding environment with AI superpowers bolted on. If you're already comfortable (or getting comfortable) working in a code editor, Cursor feels natural.

Standout feature: The agent sees your whole project context. When you say "add authentication," it knows about your existing routes, your database setup, and your frontend structure. It doesn't work in a vacuum.

Claude Code

What it is: Anthropic's terminal-based AI coding agent. You run it from your command line, and it has full access to your file system, can run any terminal command, and works through complex multi-step tasks autonomously.

How it works: You open your terminal, navigate to your project folder, and start a conversation with Claude Code. It can read any file in your project, create new files, edit existing ones, run npm install, execute tests, commit to git — basically anything you could do in a terminal.

Best for: Developers (and ambitious non-coders) who want maximum power and don't mind working in a terminal. It's incredible for backend work, complex refactors, and tasks that require running commands.

Standout feature: Full terminal access means it can do things other agents can't — run your app, check for errors in real-time, install packages, and even deploy. It's the most "autonomous" of the mainstream options.

Lovable

What it is: A browser-based AI agent that generates full-stack web applications from natural language descriptions. You describe what you want, and Lovable builds it — frontend, backend, database, and all.

How it works: You go to Lovable's website, describe your app in plain English ("Build me a task management app with user accounts, project boards, and due date reminders"), and it generates a complete React + Supabase application. You can then iterate on it through conversation: "Make the sidebar collapsible" or "Add a dark mode toggle."

Best for: Non-coders who want to go from idea to working app as fast as possible. The visual, browser-based interface means you never have to touch a terminal or code editor if you don't want to.

Standout feature: Instant full-stack generation. Other tools help you write code — Lovable generates entire applications, including the database schema and authentication system.

Replit

What it is: A cloud-based IDE (code editor that runs in your browser) with an integrated AI agent. You can write, run, and deploy code without installing anything on your computer.

How it works: You create a new project in Replit's browser-based editor. The AI agent can generate code, explain existing code, debug errors, and help you deploy your app — all within the same browser tab. Everything runs in the cloud, so you don't need a powerful computer.

Best for: Beginners who want zero setup friction. You can go from "I have an idea" to "my app is live on the internet" without ever leaving your browser.

Standout feature: Instant deployment. Your app is already running in the cloud by default. Hit "Deploy" and it's live with a real URL. No configuring servers, no deployment pipelines.

GPT Codex

What it is: OpenAI's cloud-based coding agent. You give it a task (or point it at a GitHub issue), and it spins up a sandboxed cloud environment, writes and tests the code, and delivers the results.

How it works: You describe a task or link a GitHub issue. Codex creates an isolated cloud environment with your codebase, works through the implementation, runs tests to verify its work, and presents you with the changes. You review and merge.

Best for: Teams that want an asynchronous coding assistant — you fire off tasks and come back later to review completed work. Great for knocking out a backlog of small-to-medium tasks.

Standout feature: Cloud sandboxing means it can run code, install packages, and test everything in an isolated environment. It won't break your local setup because it never touches it.

Windsurf

What it is: An agentic code editor built by Codeium. Like Cursor, it's a full IDE, but it's built from the ground up around AI-first workflows rather than being AI added to an existing editor.

How it works: Windsurf provides what it calls "Cascade" — an agentic flow that can read your codebase, plan changes, make multi-file edits, and run terminal commands. It combines the chat and agent paradigms into a single, fluid experience.

Best for: People who want an alternative to Cursor with a different approach to the agent experience. Windsurf puts a strong emphasis on understanding your full codebase context.

Standout feature: Deep codebase indexing means the agent often has better context about your project than you do. It can find relevant files and patterns you didn't even know existed.

Bolt (by StackBlitz)

What it is: A browser-based AI agent that can generate and run full web applications entirely in your browser using WebContainers technology.

How it works: Similar to Lovable, you describe what you want and Bolt generates it. The difference is that Bolt runs the entire development environment (including Node.js) directly in your browser tab using StackBlitz's WebContainers. No cloud server needed.

Best for: Quick prototyping and experimentation. Want to see an idea come to life in 60 seconds? Bolt is hard to beat.

Standout feature: Everything runs in-browser. There's no server spinning up in the cloud — the Node.js environment runs right in your browser tab, which makes it incredibly fast to iterate.

Devin (by Cognition)

What it is: Positioned as the "first AI software engineer," Devin is a fully autonomous agent designed to handle complete software engineering tasks end-to-end with minimal human intervention.

How it works: You assign Devin a task (via Slack, their web interface, or other integrations), and it goes to work. It has its own virtual computer with a browser, code editor, and terminal. It can research solutions, write code, test it, debug issues, and even interact with external services.

Best for: Enterprise teams with repetitive engineering tasks or a large backlog. Devin is designed for longer, more complex tasks that would take a human engineer hours or days.

Standout feature: True autonomy. Devin doesn't need hand-holding — you can assign a task and walk away. It will ask questions only when it genuinely gets stuck.

The Big Comparison Table

Here's the comparison you've been waiting for:

Platform	Type	Price Range	Best For	Key Limitation
Cursor	Desktop IDE	Free tier; Pro ~\$20/mo	Daily coding with AI assistance	Requires local setup, learning curve for IDE
Claude Code	Terminal agent	Included with Claude subscription	Complex multi-step tasks, backend work	Terminal-based (intimidating for beginners)
Lovable	Browser app builder	Free tier; paid plans from ~\$20/mo	Non-coders building full apps	Less control over code details
Replit	Cloud IDE	Free tier; paid plans from ~\$25/mo	Zero-setup development and deployment	Performance can lag on free tier
GPT Codex	Cloud agent	Included with ChatGPT Pro (\$200/mo)	Async task completion, GitHub integration	Expensive, requires ChatGPT Pro plan
Windsurf	Desktop IDE	Free tier; Pro plans available from ~\$20/mo	Full IDE experience with agentic AI	Newer platform, smaller community
Bolt	Browser agent	Free tier; paid plans from ~\$20/mo	Quick prototypes, visual projects	Less suited for complex backends
Devin	Autonomous agent	Enterprise pricing (~\$500+/mo)	Enterprise teams, complex autonomous tasks	Very expensive, can be slow on large tasks

How to Choose the Right Tool

Feel overwhelmed by the options? Here's a simple decision framework:

Are you a complete beginner who wants the fastest path to a working app?

! Start with Lovable or Bolt. Browser-based, no setup, instant results.

Are you comfortable with a code editor and want AI integrated into your workflow?

! Go with Cursor or Windsurf. Best balance of control and AI assistance.

Do you want maximum power and don't mind the terminal?

! Claude Code is your tool. Unmatched autonomy and system access. You can now download this as a desktop app on the Mac too as of Feb 2026.

Do you want everything in the cloud with zero local setup?

! Replit gives you a full IDE, hosting, and AI in your browser.

Are you on a team and want to assign tasks to AI asynchronously?

! Look at GPT Codex or Devin for task-based autonomous work.

Not sure at all?

! Start with Cursor (free tier) or Claude Code. If you're on a Mac use the desktop app or on the web. It's the most popular option, has the largest community, and strikes the best balance between power and accessibility.

The Future of AI Agents in Development

We're still in the early days. Here's where things are heading:

Agents will get more autonomous. Today, you still need to review and approve most changes. In the near future, agents will handle increasingly complex tasks with less oversight — think "build this entire feature and deploy it to staging" as a single instruction.

Specialization will increase. We'll see agents purpose-built for specific tasks: one for database design, one for UI/UX, one for testing, one for security auditing. You'll orchestrate a team of agents.

The barrier to entry will keep dropping. Every month, these tools get easier to use. The gap between "I have an idea" and "I have a working app" is shrinking rapidly. Non-coders building production applications is not a future prediction as it's happening right now.

Collaboration between humans and agents will improve. Better interfaces for reviewing agent work, clearer explanations of what agents did and why, and more granular control over agent autonomy. The goal isn't to remove humans from the process, it's to make humans dramatically more effective.

Key Takeaways

- There's no single "best" AI agent — each platform has strengths. Cursor and Claude Code offer the most control; Lovable and Bolt offer the fastest path to a working app; GPT Codex and Devin offer the most autonomy.
- Browser-based tools (Lovable, Bolt, Replit) are best for beginners. IDE-based tools (Cursor, Windsurf) are best for ongoing development. Terminal-based tools (Claude Code) offer maximum power.
- Cost varies dramatically — from free tiers on most platforms to \$500+/month for enterprise agents like Devin. Start free and upgrade when you hit limits.
- The landscape is evolving fast. The tool that's best today might be leapfrogged tomorrow. Stay flexible, experiment often, and don't get too attached to any single platform.
- Focus on what you're building, not what you're building with. The best tool is the one that helps you ship your project. Period.